

Object Inheritance Lab Part Two (CodeGrade)

 <https://github.com/learn-co-curriculum/python-p3-inheritance-lab-pt-2>  <https://github.com/learn-co-curriculum/python-p3-inheritance-lab-pt-2/issues/new>

Learning Goals

- Reduce repeated code and enhance objects using inheritance.
 - Define two classes, one of which inherits from another.
 - Define methods in the child class that use the `super` keyword to inherit from and augment methods in the parent class.
 - Accomplish complex programming tasks using knowledge from previous modules.
-

Key Vocab

- **Inheritance**: a tool that allows us to recycle code by creating a class that "inherits" the attributes and methods of a parent class.
 - **Composition**: a tool that enables you to recycle code by adding objects to other objects. Rather than building on a base class as in inheritance, composition leverages the attributes and methods of an instance of another class.
 - **Subclass**: a class that inherits from another class. Colloquially called a "child" class.
 - **Superclass**: a class that is inherited by another class. Colloquially called a "parent" class.
 - **Child**: another name for a subclass.
 - **Parent**: another name for a superclass.
 - `super()`: a built-in Python function that allows us to manipulate the attributes and methods of a superclass from the body of its subclass.
 - **Decorator**: syntax that allows us to add functionality to an object without modifying its structure.
-

Instructions

In this lab, you'll be coding a `Student` class, which will be the superclass, and a `ChattyStudent` class, which will inherit from `Student`.

`ChattyStudent` is a student, so they should have all of the behaviors and characteristics of a student. The difference is that `ChattyStudent` is *very chatty*. So, they will elaborate on any phrases that are inherited from `Student`.

Run the test suite to get started. This is a test-driven lab.

1. Write a method in the `Student` class, `hello()`, that `print()`s out the phrase:

"Hey there! I'm so excited to learn stuff."

1. Write a method in the `Student` class, `raise_hand()`, that `print()`s out the phrase:

"Pick me!"

1. Write a method in the `ChattyStudent` class, `hello()`, that uses `super()` to inherit the behavior of the `hello()` method from the parent class. Then, augment that method to also `print()` out the very chatty phrase:

"How are you doing today? I'm okay, but I'm kind of tired. Did you watch The Walking Dead last night? You didn't?! Oh man, it was so crazy! What, you don't want any spoilers? Okay well let me just tell you who died..."

1. Write a method in the `ChattyStudent` class, `raise_hand()`, that uses `super()` ten times so that the method will `print()` out *"Pick me!"* ten times. **It is possible to call `super()` multiple times in a method.**

Resources

- [Python 3.8 Documentation](https://docs.python.org/3.8/)  [\(https://docs.python.org/3.8/\)](https://docs.python.org/3.8/)
- [Inheritance - Python](https://docs.python.org/3/tutorial/classes.html#inheritance)  [\(https://docs.python.org/3/tutorial/classes.html#inheritance\)](https://docs.python.org/3/tutorial/classes.html#inheritance)
- [Inheritance and Composition: A Python OOP Guide - Real Python](https://realpython.com/inheritance-composition-python/)  [\(https://realpython.com/inheritance-composition-python/\)](https://realpython.com/inheritance-composition-python/)
- [Decorators in Python - GeeksforGeeks](https://www.geeksforgeeks.org/decorators-in-python/)  [\(https://www.geeksforgeeks.org/decorators-in-python/\)](https://www.geeksforgeeks.org/decorators-in-python/)

- [Supercharge Your Classes With Python super\(\) - Real Python](https://realpython.com/python-super/)  (<https://realpython.com/python-super/>)

This tool needs to be loaded in a new browser window

Load Object Inheritance Lab Part Two (CodeGrade) in a new window